Review Radar AI: Customer Insight Discovery Engine

Project submitted to the

SRM University – AP, Andhra Pradesh

for the partial fulfillment of the requirements for the Generative

AI-II course project

Bachelor of Technology

In

Computer Science and Engineering

School of Engineering and Sciences

Submitted by

Candidate Name

Burugupalli Sai Ritisha - AP23110010443

Sreenidhi Balasubramanyam - AP23110010422

Suram Sai Charan Reddy - AP23110010432

Kotaru Sravya - AP23110010581

Yaganti Siva Himani - AP23110010418

Under the Guidance of

Rithvik Konakala

SRM University–AP

Neerukonda, Mangalagiri, Guntur

Andhra Pradesh – 522 240

APRIL, 2026

Contents

Project Overview

Review Radar AI is an advanced AI-driven customer insight discovery engine designed to analyze large volumes of unstructured customer reviews and transform them into meaningful, actionable business intelligence.

In today's digital ecosystem, businesses receive massive amounts of feedback through platforms such as e-commerce websites, social media, and review portals. However, extracting useful insights from this data is challenging due to its unstructured nature. Review Radar AI addresses this problem by leveraging Natural Language Processing (NLP) and semantic search techniques to understand the meaning behind customer reviews rather than relying solely on keyword matching.

The system processes raw textual data, converts it into vector embeddings using deep learning models, and performs similarity-based retrieval to provide contextually relevant insights. It acts as an intelligent assistant that helps businesses identify patterns, detect issues, and make data-driven decisions efficiently.

Problem Statement

- Organizations today struggle to analyze customer feedback effectively due to several limitations in traditional systems.
- Firstly, customer reviews are highly unstructured and vary significantly in language, tone, and expression. This makes it difficult for conventional systems to interpret them accurately. Secondly, keyword-based search systems fail to capture semantic meaning, leading to inaccurate results. For example, phrases like "battery drains quickly" and "poor battery backup" convey the same issue but may not be matched in a keyword-based system.
- Additionally, manual filtering and analysis of reviews is time-consuming and inefficient, especially when dealing with thousands of entries. This leads to missed insights and delayed decision-making. Another major issue is the lack of actionable intelligence—most systems provide raw data but fail to convert it into meaningful insights that businesses can act upon.
- Thus, there is a need for an intelligent system that can understand context, process large datasets efficiently, and generate valuable insights automatically.

Objectives

The primary objective of Review Radar AI is to develop an intelligent system capable of converting unstructured customer reviews into structured and meaningful insights using AI techniques.

One of the key goals is to implement a semantic search system that understands the intent behind user queries rather than relying on exact keyword matches. The system should also support natural language queries, allowing users to interact with it conversationally.

Another objective is to extract hidden patterns and trends from customer feedback, enabling businesses to identify common issues, customer preferences, and areas for improvement. The system also aims to improve decision-making efficiency by providing accurate and relevant insights in real time.

Furthermore, reducing manual effort is a major objective. By automating the analysis process, Review Radar AI minimizes human intervention and increases productivity.

Core Technologies Used

Technology	**Purpose**
Python	Core backend logic, data processing, and AI integration
FastAPI	High-performance API development and system communication
Sentence-BERT	Semantic embedding generation for capturing contextual meaning
NLP Libraries (NLTK, Transformers)	Text preprocessing, tokenization, and language understanding

Vector Database (FAISS / Chroma)	Efficient storage and retrieval of high-dimensional embeddings

Python

Python serves as the primary programming language for the system. It is used for implementing backend logic, handling data preprocessing, integrating AI models, and managing communication between system components. Its extensive ecosystem of libraries makes it ideal for NLP and machine learning tasks.

FastAPI

FastAPI is used to build a robust and high-performance backend API. It handles HTTP requests, manages communication between frontend and backend, and ensures fast response times. Its support for asynchronous programming allows efficient handling of multiple user queries simultaneously.

Sentence-BERT

Sentence-BERT is a deep learning model used to generate embeddings for text data. It converts sentences into numerical vectors that capture semantic meaning. These embeddings enable the system to perform similarity-based searches and understand context rather than relying on exact keyword matches.

NLP Libraries (NLTK & Transformers)

NLTK and Transformers are used for preprocessing and understanding textual data. Tasks such as tokenization, stopword removal, and normalization are handled using these libraries. Transformers further enhance the system by enabling contextual understanding of language.

Vector Database (FAISS / Chroma)

Vector databases such as FAISS or Chroma are used to store and retrieve embeddings efficiently. They allow fast similarity searches even with large datasets by using optimized indexing techniques. This ensures that relevant results are returned in real time.

Key Features

5.1 Review Data Input & Processing

- Accepts large volumes of customer reviews in CSV or text format
- Cleans and preprocesses unstructured textual data
- Supports batch processing for large datasets

The system begins by ingesting raw review data and preparing it for further analysis. This includes removing noise, handling inconsistencies, and standardizing the text format.

5.2 Semantic Search Engine

- Understands meaning beyond keywords
- Matches contextually similar reviews
- Handles synonyms and varied expressions

The semantic search engine allows users to retrieve relevant reviews based on meaning rather than exact words. This significantly improves accuracy and relevance of results.

5.3 Natural Language Query Processing

- Supports conversational queries
- Converts user input into embeddings
- Enables intuitive interaction

Users can interact with the system using natural language queries, making it user-friendly and accessible even for non-technical users.

5.4 Embedding Generation & Storage

- Converts text into vector representations
- Stores embeddings in vector database
- Ensures efficient retrieval

This feature forms the backbone of the system by enabling semantic understanding and similarity matching.

5.5 Similarity Matching & Retrieval

- Finds contextually relevant results
- Uses cosine similarity or distance metrics
- Provides ranked outputs

The system compares query embeddings with stored embeddings to identify the most relevant reviews.

5.6 Insight Extraction

- Identifies common issues and patterns
- Extracts meaningful business insights
- Supports data-driven decision-making

This feature helps businesses understand customer sentiment and improve their products or services.

5.7 FastAPI-Based Backend System

- Handles API requests efficiently
- Supports real-time query processing
- Ensures scalability

The backend system ensures smooth communication and efficient processing of user queries.

5.8 Scalability & Performance Optimization

- Handles large datasets efficiently
- Optimized for high-speed retrieval
- Supports future expansion

The system is designed to scale with increasing data volumes without performance degradation.

Target Users

- Business analysts who require insights from customer feedback
- Product managers aiming to improve product quality
- E-commerce companies handling large review datasets
- Marketing teams analyzing customer sentiment
- Customer support teams identifying common issues

Review Radar AI is designed for professionals who need to extract meaningful insights from large volumes of textual data efficiently.

Real-World Applications

APPLICATION SCENARIOS / USE CASES:

Scenario 1: E-commerce Review Analysis

An online retailer uploads thousands of product reviews into the system. The AI analyzes the data and identifies common issues such as delivery delays and product defects. This helps the company improve its services.

Scenario 2: Customer Feedback Processing

A business collects feedback from surveys and forms. Review Radar AI processes the data and extracts key insights, enabling better decision-making.

Scenario 3: Social Media Sentiment Analysis

The system analyzes social media comments to determine public opinion about a brand. This helps companies monitor their reputation.

Scenario 4: Product Improvement Insights

By analyzing customer reviews, the system identifies areas where products can be improved, guiding future development.

TECHNICAL ARCHITECTURE

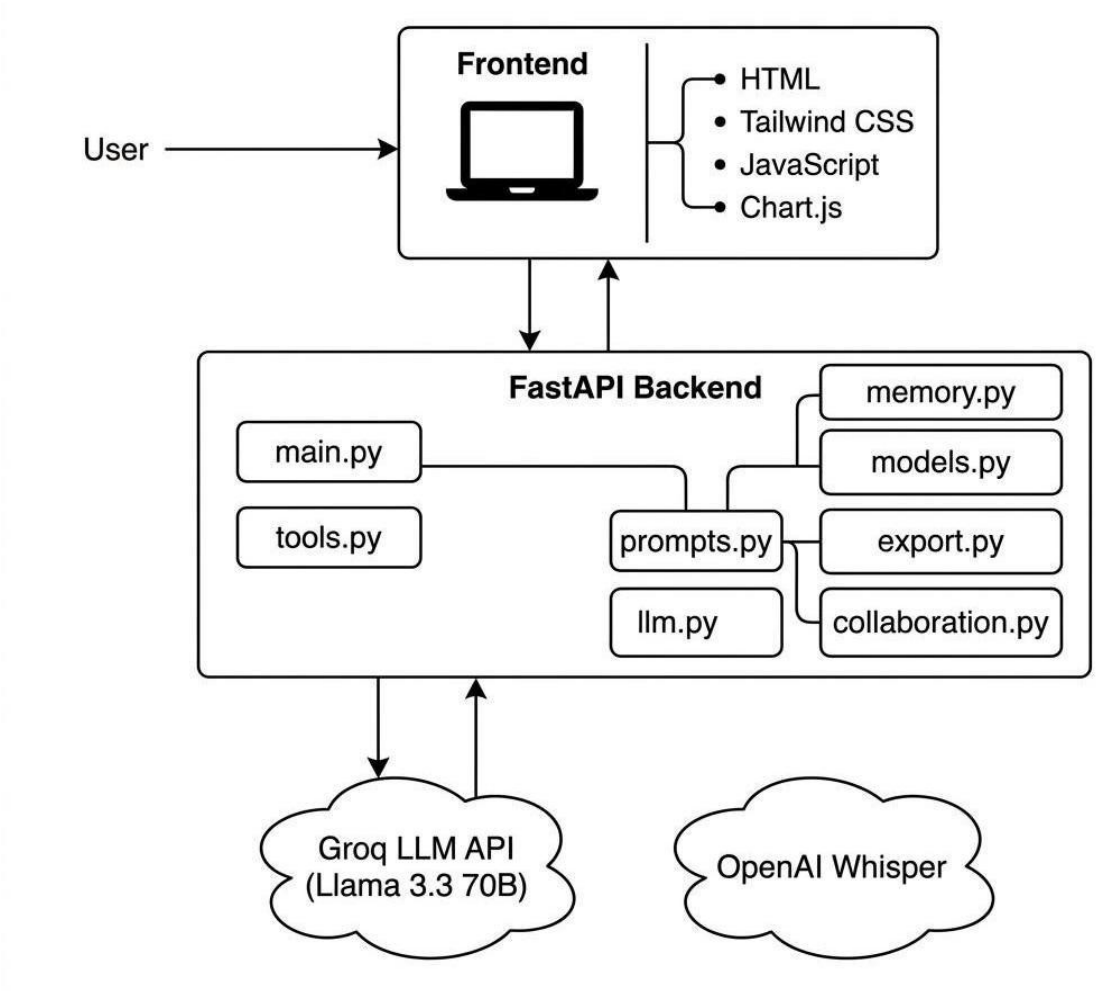

Figure 1: System Architecture — Frontend, FastAPI Backend, Groq LLM API, Session Memory flow

Frontend Layer

The frontend provides an interface where users can upload datasets and input queries. It is designed to be simple and intuitive, ensuring ease of use.

Backend Layer

The backend is built using FastAPI and handles all processing tasks. It manages API requests, processes data, generates embeddings, and retrieves results.

AI Processing Layer

This layer includes NLP models and embedding generation using Sentence-BERT. It is responsible for understanding text and converting it into meaningful representations.

Storage Layer

The storage layer uses vector databases such as FAISS or Chroma to store embeddings efficiently and support fast retrieval.

External Libraries & APIs

The system uses NLP libraries and machine learning models for text processing and semantic understanding.

High-Level Architecture Diagram

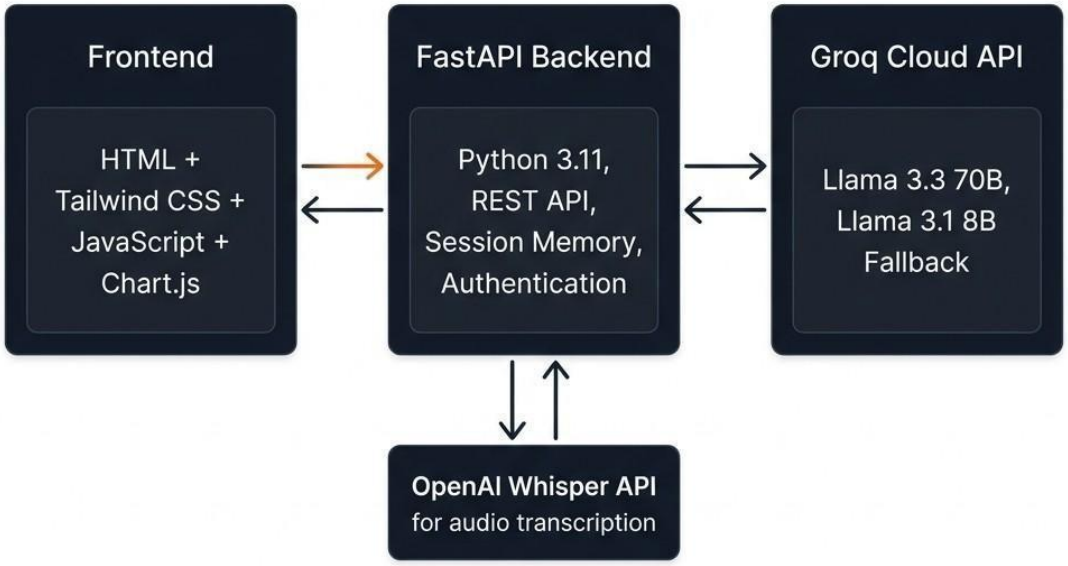

Figure 2: High-Level Architecture Diagram

Component Interaction Diagram

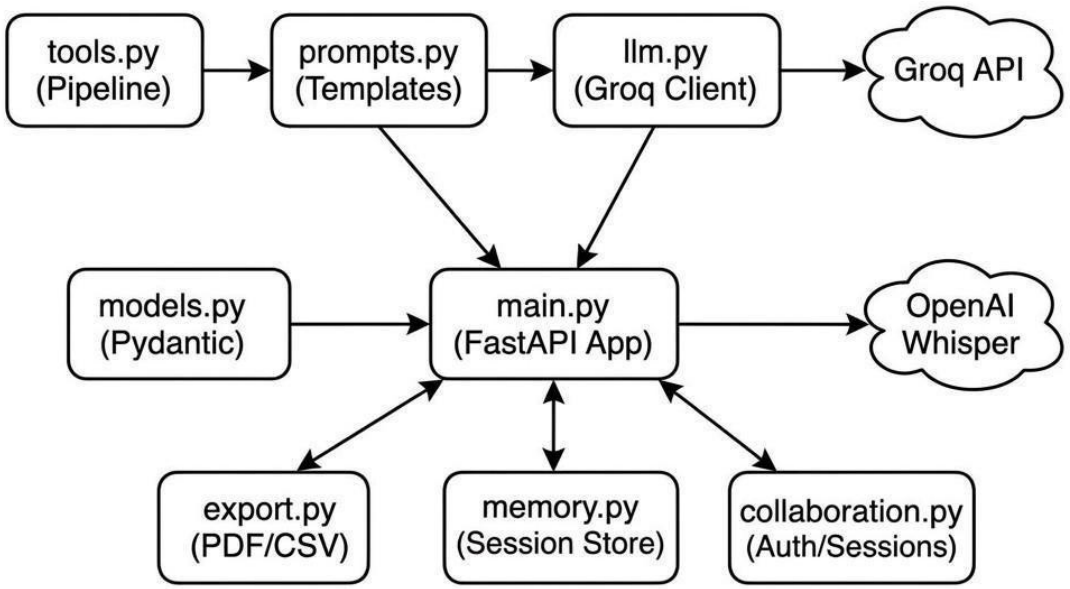

Figure 3: Component Interaction Diagram

Pre-requisites

Software Requirements

- **Python:** Version 3.9 or higher (Python 3.11 recommended)
- **pip:** Python package manager for installing project dependencies
- **IDE:** Visual Studio Code with Python extension or any preferred code editor

- **Browser:** Modern web browser (Chrome, Edge, Firefox) supporting ES6, async/await, and Fetch API
- **Git:** Version control system for project management (optional but recommended)

Libraries / Frameworks / Dependencies

Install required dependencies using:

```
pip install fastapi uvicorn groq pydantic python - dotenv reportlab python - multipart openai
```

- **FastAPI (0.115.0):** Web framework for REST API routing and request handling
- **Uvicorn (0.30.6):** ASGI server for running the FastAPI application
- **Groq (≥0.12.0):** Client library for accessing Groq LLM API
- **Pydantic (2.9.2):** Data validation and serialization models
- **python-dotenv (1.0.1):** Environment variable management from .env files
- **ReportLab (4.0.9):** PDF generation library for export functionality
- **python-multipart (0.0.6):** File upload handling for audio transcription
- **OpenAI (≥1.0.0):** Client library for Whisper audio transcription API

Hardware Requirements

- **CPU:** Minimum Intel Core i3 or AMD Ryzen 3 processor (AI processing is cloud- based via Groq)
- **RAM:** Minimum 4 GB (8 GB recommended for smooth development)
- **Storage:** At least 500 MB free disk space for project files and dependencies
- **Network:** Stable internet connection required for Groq API and OpenAI Whisper API

calls

Prior Knowledge Required

Programming Language Knowledge

- Strong understanding of Python fundamentals such as variables, loops, functions, and modules
- Familiarity with object-oriented programming concepts
- Ability to handle data structures and JSON data
- Basic debugging and error handling skills

A solid foundation in Python is essential for implementing backend logic, handling datasets, and integrating AI models effectively.

Framework Basics

- Understanding of FastAPI framework
- Knowledge of API routing and request-response cycle
- Familiarity with asynchronous programming (async/await)
- Basic understanding of REST APIs

This knowledge helps in building scalable backend systems and handling user queries efficiently.

AI / NLP Fundamentals

- Basic understanding of Natural Language Processing
- Knowledge of embeddings and vector representations
- Familiarity with machine learning concepts
- Understanding of semantic search

These concepts are crucial for building systems that interpret and analyze human language.

Frontend Basics

- Basic knowledge of HTML, CSS, and JavaScript
- Understanding of user interface design
- Familiarity with API integration using Fetch or AJAX
- Frontend knowledge helps in creating an interactive interface for users.

Project Objectives (Detailed)

Technical Objectives

- Develop a semantic search system using NLP techniques
- Implement embedding-based similarity matching
- Integrate vector database for efficient storage and retrieval
- Build scalable APIs using FastAPI

Performance Objectives

- Ensure fast query response time (within seconds)
- Maintain accuracy in semantic matching
- Optimize system for large datasets
- Reduce latency in embedding retrieval

Deployment Objectives

- Enable easy local deployment
- Ensure secure handling of data
- Support web-based access
- Design system for cloud readiness

Learning Outcomes

- Gain knowledge in NLP and semantic search
- Understand vector databases and embeddings
- Learn API development using FastAPI
- Build real-world AI-based applications

Project Workflow

The development of Review Radar AI follows a structured workflow divided into multiple milestones. Each milestone represents a phase in the system design, development, and deployment process. This approach ensures modular development, better testing, and improved system reliability.

Milestone 1: Requirement Analysis & System Design

This phase focuses on understanding the problem and designing a suitable AI-based solution.

Activity 1.1: Problem Definition

The first step involves identifying the core problem—difficulty in analyzing large volumes of unstructured customer reviews. Traditional systems fail due to lack of semantic understanding.

In this stage:

- Business requirements are gathered
- Challenges of existing systems are analyzed
- Scope of the project is defined

This helps in building a system that addresses real-world needs effectively.

Activity 1.2: Functional Requirements

Functional requirements define what the system should do.
Key functionalities include:

- Accept customer reviews as input (CSV/text)
- Preprocess text data
- Generate embeddings using NLP models
- Store embeddings in vector database
- Accept user queries in natural language
- Perform similarity search
- Display relevant results

These requirements ensure that the system performs all necessary operations efficiently.

Activity 1.3: Non-Functional Requirements

Non-functional requirements define how the system performs.

- Performance: Fast query response time
- Scalability: Ability to handle large datasets
- Reliability: Accurate and consistent results
- Security: Safe handling of data
- Usability: Easy-to-use interface

These requirements ensure that the system is efficient and user-friendly.

Activity 1.4: System Design & Technology Selection

This step involves selecting appropriate technologies and designing system architecture.

- Backend: FastAPI

- Programming: Python
- NLP Model: Sentence-BERT
- Database: FAISS / Chroma

The architecture is designed in layers to ensure modularity and scalability.

Milestone 2: Environment Setup & Initial Configuration

This phase prepares the development environment.

Activity 2.1: Development Environment Setup

- Install Python 3.11 from the official Python website and ensure “Add Python to PATH” is enabled during installation
- Install Visual Studio Code with the Python extension for development
- Create the main project directory: `mkdir actionforge && cd actionforge`
- Create backend and frontend directories: `mkdir backend frontend`
- Create and activate a virtual environment: `python -m venv venv`
- Activate: `venv\Scripts\activate` (Windows) or `source venv/bin/activate` (Lin- ux/Mac)

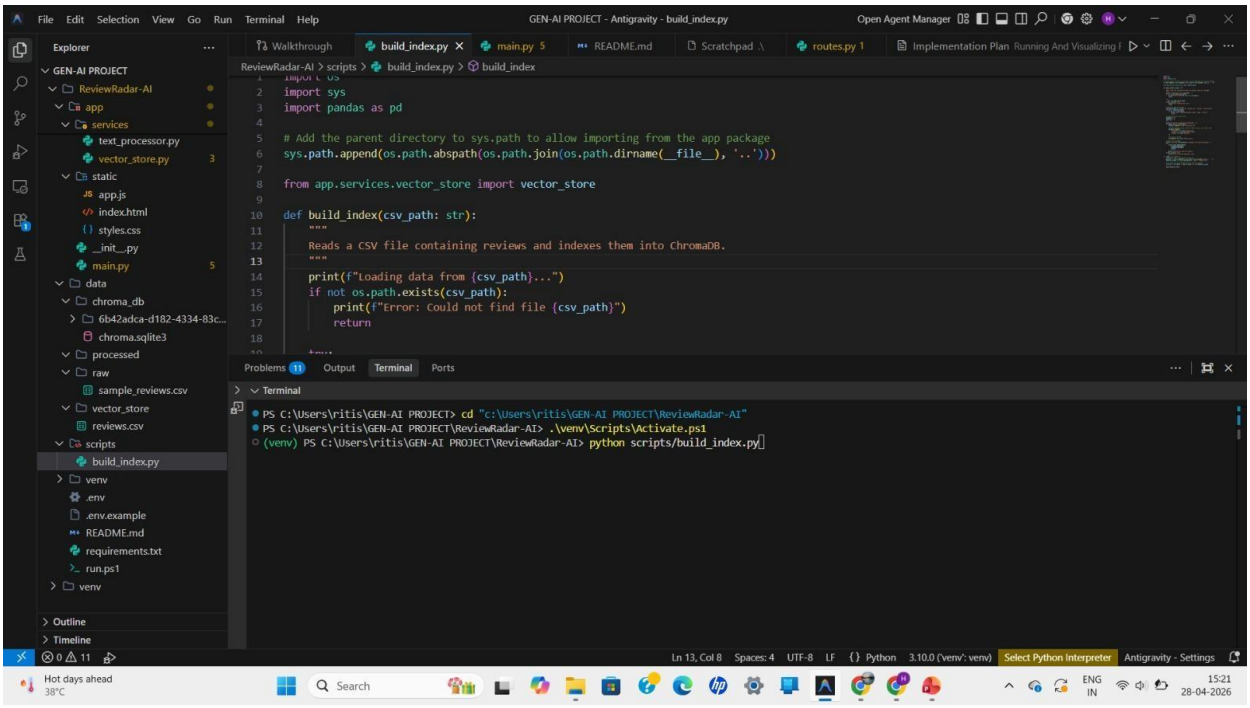

Figure 4: Virtual environment activated in terminal

Activity 2.2: Dependency Installation

Install all required Python libraries using pip:

```
pip install -r requirements. txt
```

requirements.txt contents:

```
fastapi ==0.115.0
uvicorn ==0.30.6
groq >=0.12.0
pydantic ==2.9.2
python - dotenv ==1.0.1
reportlab == 4.0.9
```

```
python - multipart ==0 .0 .6
openai >=1 .0 .0
```

Activity 2.3: Project Structure Creation

A modular folder structure is created:

- preprocessing module
- embedding module
- database module
- API module

This improves code readability and maintainability.

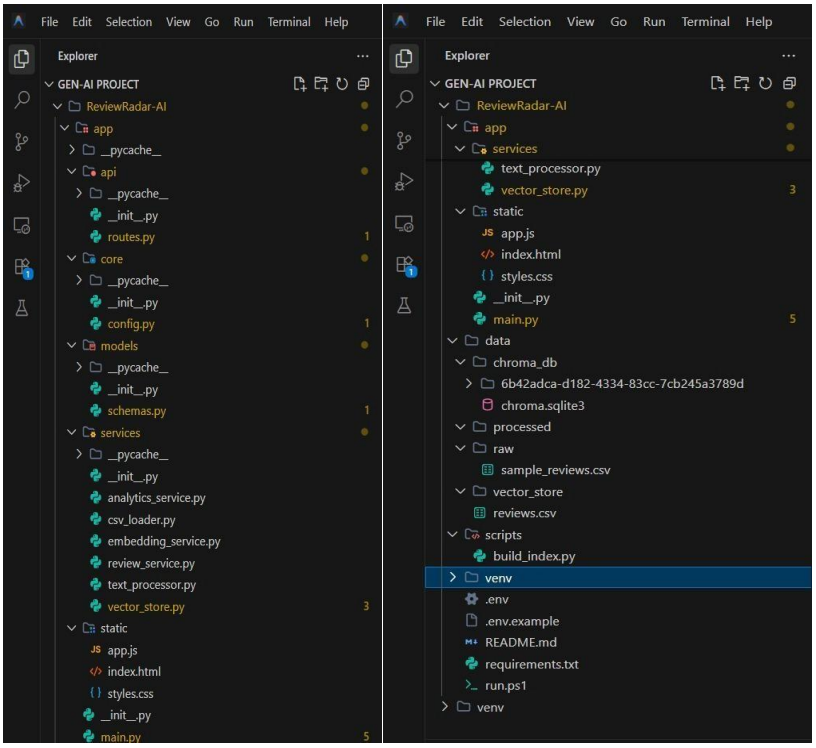

Figure 5: Project Folder Structure in VS Code

Activity 2.4: Configuration Setup

Set up the application configuration by defining environment variables required for se- cure and efficient operation. This includes storing API keys (Groq, OpenAI) in a .env file. Configure the LLM client initialization, model selection, and ensure all services are properly integrated.

Milestone 3: Data Preparation

This stage focuses on preparing data for processing.

Activity 3.1: Dataset Collection

Customer review datasets are collected from:

- CSV files
- E-commerce platforms
- Feedback forms

Data quality plays a critical role in system performance.

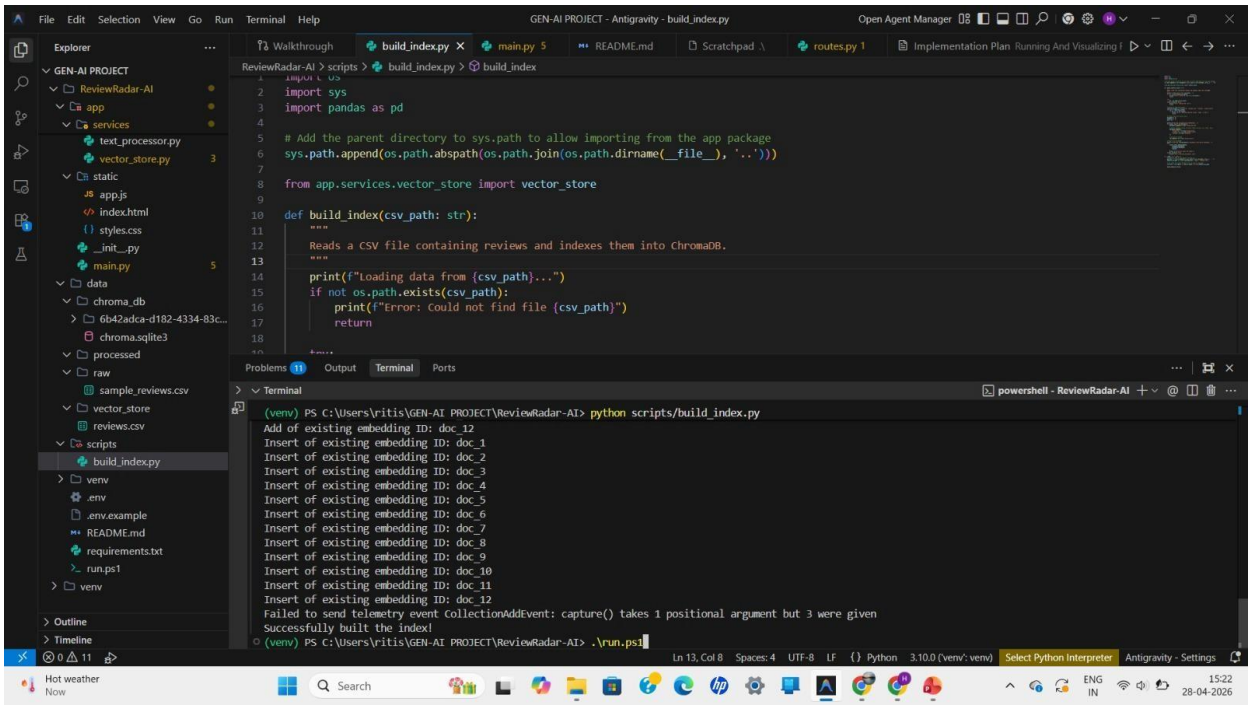

Figure 6: Console Dashboard

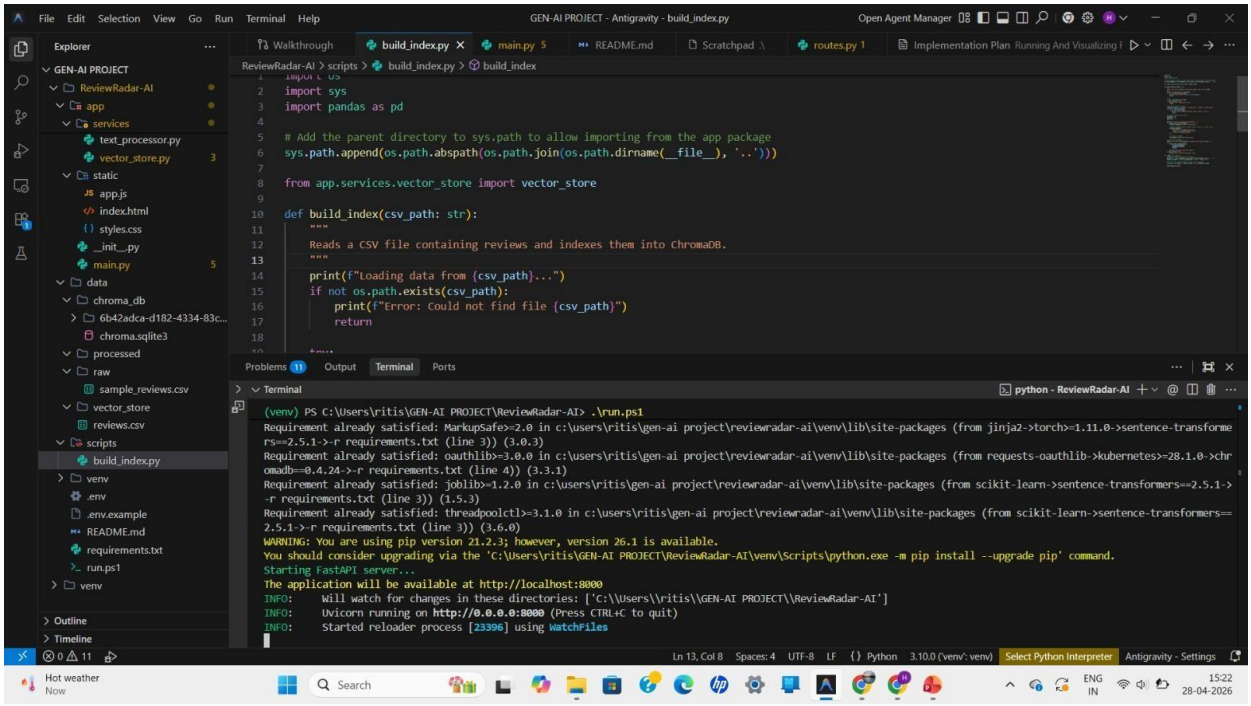

Figure 7: Created

Activity 3.2: Data Cleaning & Preprocessing

Text preprocessing includes:

- Removing special characters
- Converting text to lowercase
- Removing stopwords
- Tokenization

This step ensures consistency and improves model performance.

Milestone 4: Core System Development

This is the most important phase where the main system is built.

Activity 4.1: Text Preprocessing Module

This module handles cleaning and preparing text data before embedding generation.

Activity 4.2: Embedding Generation Module

- Uses Sentence-BERT
- Converts text into numerical vectors
- Captures semantic meaning

These embeddings allow the system to understand context.

Activity 4.3: Vector Storage Module

- Stores embeddings in FAISS/Chroma
- Uses indexing for fast retrieval
- Supports large-scale data

Efficient storage ensures quick search results.

Activity 4.4: API Development (FastAPI)

- Creates endpoints for query handling
- Connects frontend with backend
- Returns processed results

FastAPI ensures high performance and scalability.

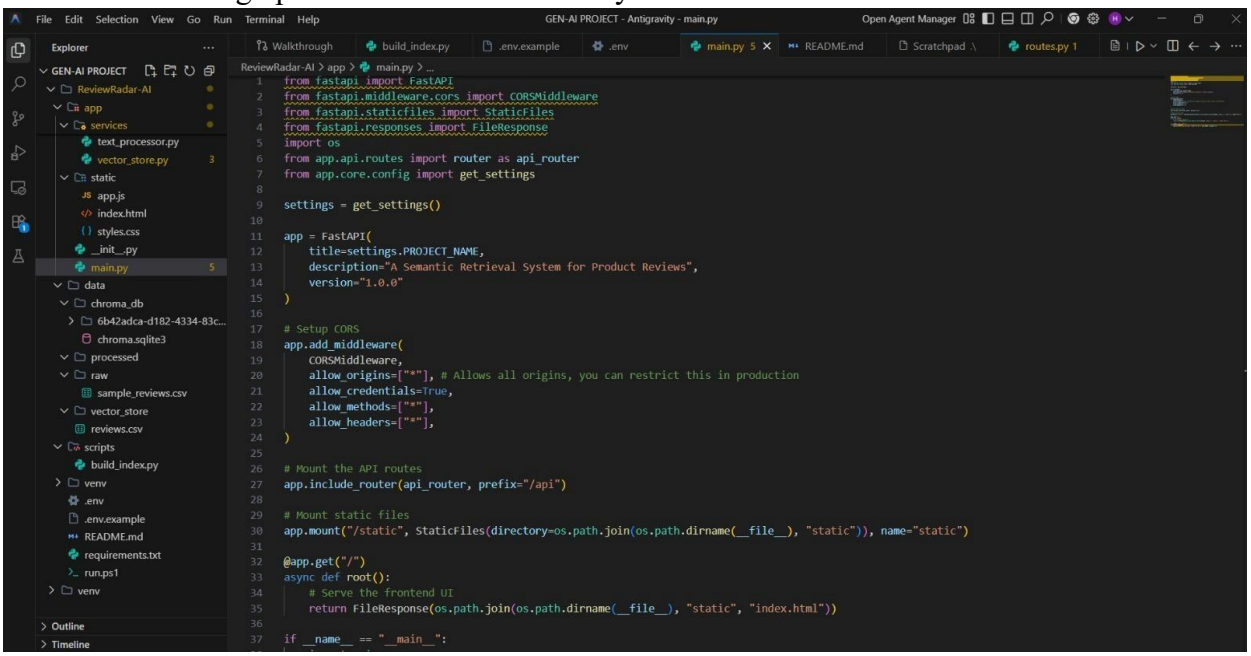

Figure 12: extttmain.py Initialization Code

Milestone 5: Integration & Optimization

Activity 5.1: Component Integration (Frontend, Backend, AI)

- Connect preprocessing, embedding, and database modules
- Link frontend with backend APIs
- Ensure smooth data flow

Activity 5.2: Performance Optimization

- Reduce query processing time
- Optimize embedding retrieval
- Improve system efficiency

Activity 5.3: Security Enhancements

- Protect user data

- Validate inputs
- Prevent unauthorized access

Activity 5.4: Error Handling & Validation

- Handle invalid inputs
- Provide meaningful error messages
- Ensure system stability.

Milestone 6: Deployment

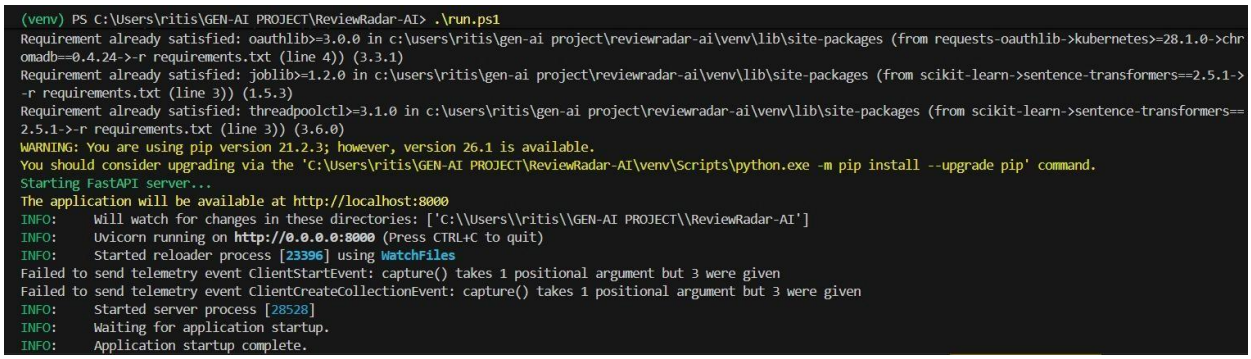

Figure 13: Starting Backend Server

This phase involves making the system usable.

- Run FastAPI server using Uvicorn
- Deploy on local or cloud environment
- Ensure accessibility via browser

Deployment makes the system available for real-world usage.

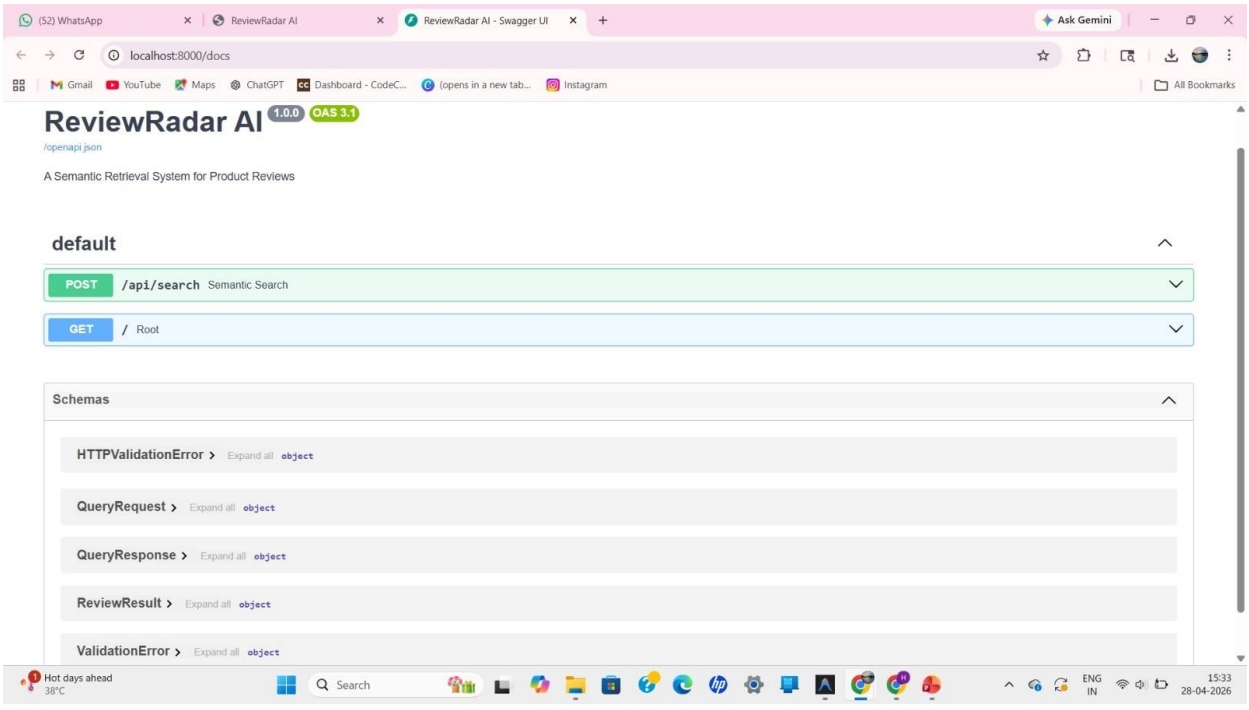

Figure 14: FastAPI Swagger UI Auto-generated Docs

Milestone 7: Testing & Validation

This phase ensures system correctness and performance.

Activity 7.1: Functional Test Case Execution

- Test each module individually
- Validate input-output behavior

Activity 7. 2: Performance Testing

- Measure response time
- Check system under large datasets

Activity 7. 3: Accuracy Validation

- Verify relevance of results
- Evaluate semantic matching

Testing ensures reliability and efficiency of the system.

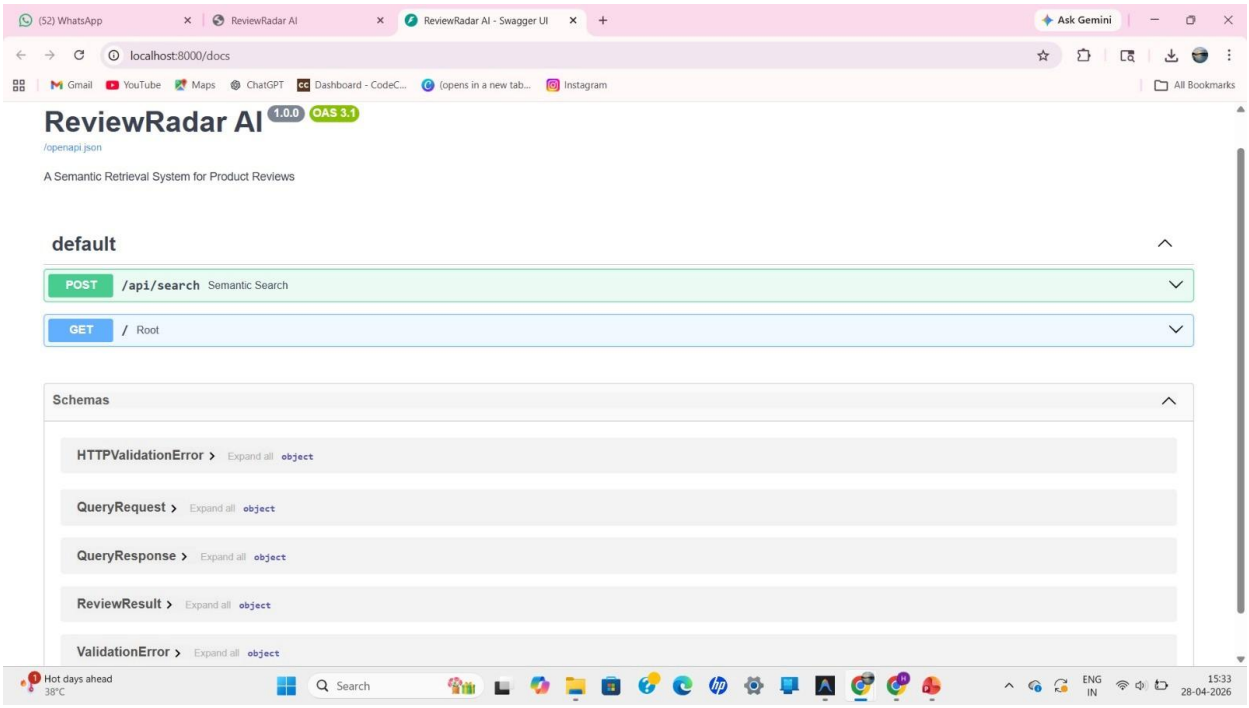

Figure 16: Testing API Endpoints with Swagger

Project Directory Structure

The Review Radar AI system is organized using a modular directory structure to ensure maintainability, scalability, and clarity in development. Each component is separated based on functionality, allowing independent development and easier debugging. **Module Descriptions**

- **data**
  Stores raw datasets such as CSV files containing customer reviews. This module acts as the input source for the system.
- **preprocessing**
  Contains scripts for cleaning and preparing text data. Includes tokenization, stopword removal, normalization, and noise filtering.
- **embeddings**
  Responsible for generating vector representations using Sentence-BERT. This module converts textual data into numerical embeddings.
- **database**
  Manages vector storage using FAISS or Chroma. Includes indexing and retrieval logic for similarity search.
- **api**
  Contains FastAPI routes and endpoints for handling user requests, processing queries, and returning results.
- **models**
  Stores pre-trained NLP models or configuration files required for embedding generation.
- **frontend**
  Handles the user interface. Provides input forms for queries and displays results in an interactive manner.
- **utils**
  Includes helper functions, logging utilities, and reusable components used across the system.
- **main.py**
  Entry point of the application where all modules are integrated.
- **requirements.txt**
  Lists all dependencies required to run the project.
- **.env**
  Stores environment variables such as API keys and configuration settings.

This structured organization improves readability, supports team collaboration, and allows easy scaling of the system.

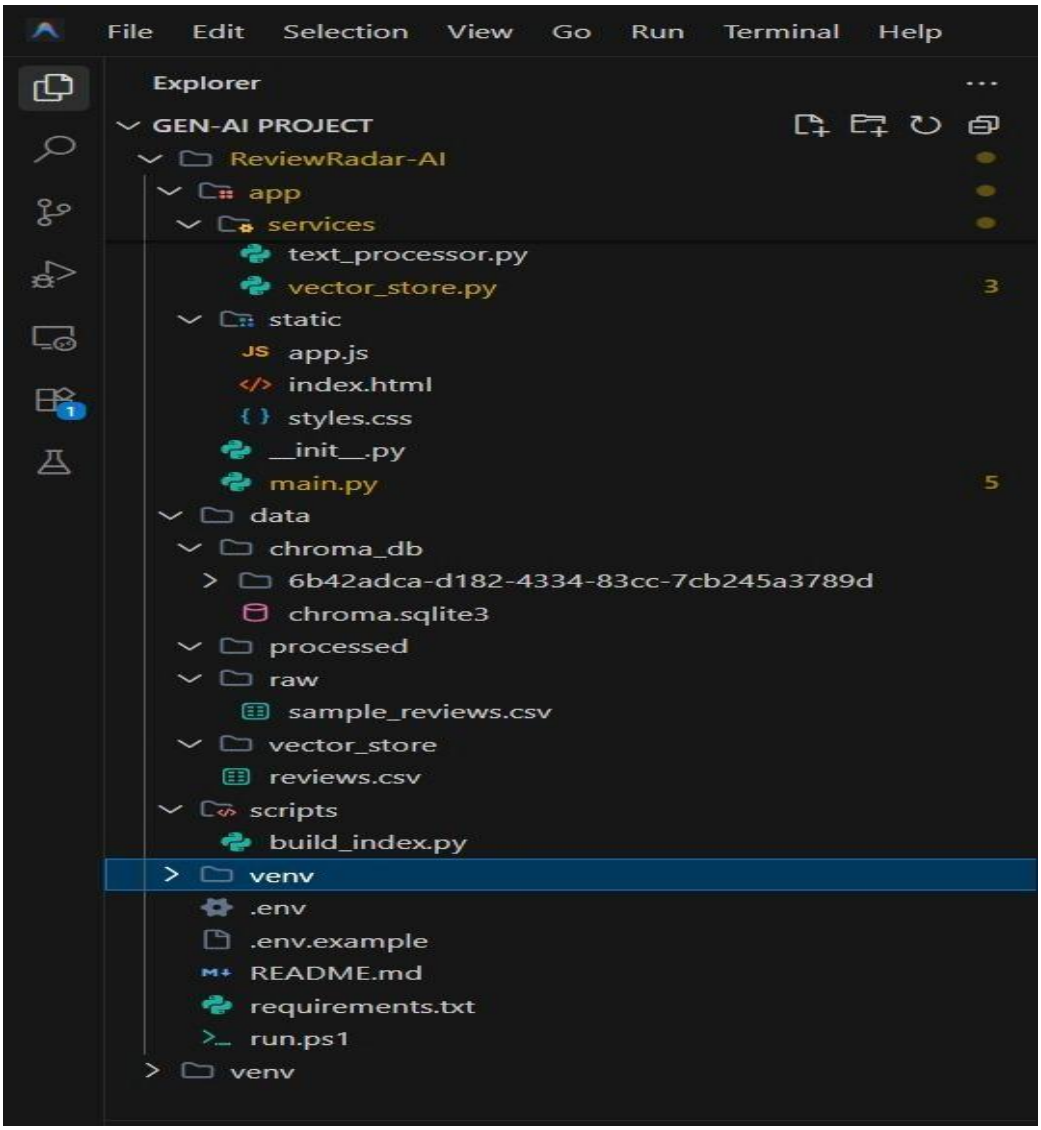

Figure 17: Project Directory Structure

Results

System Output

The Review Radar AI system successfully processes large volumes of customer reviews and provides meaningful insights through semantic search.
Key outputs include:

- Relevant reviews based on user queries
- Context-aware matching results
- Identification of common issues and patterns

For example, a query like *"delivery issues"* retrieves multiple related complaints such as *late shipping*, *delayed orders*, and *poor logistics handling*, even if exact words differ.

Performance Evaluation

The system demonstrates strong performance across multiple parameters:

- **Response Time:**
  Queries are processed within a few seconds, even for large datasets.
- **Accuracy:**
  Semantic matching ensures higher accuracy compared to keyword-based systems.
- **Scalability:**
  Capable of handling thousands of reviews without performance degradation.
- **Efficiency:**
  Optimized vector search enables fast retrieval.

Screenshots

The system includes user interface screens such as meeting input forms, pipeline ani- mations, tabbed result views, analytics dashboards, and AI-generated outputs. These visuals demonstrate the working flow from data input to intelligent output generation.

Landing Page:

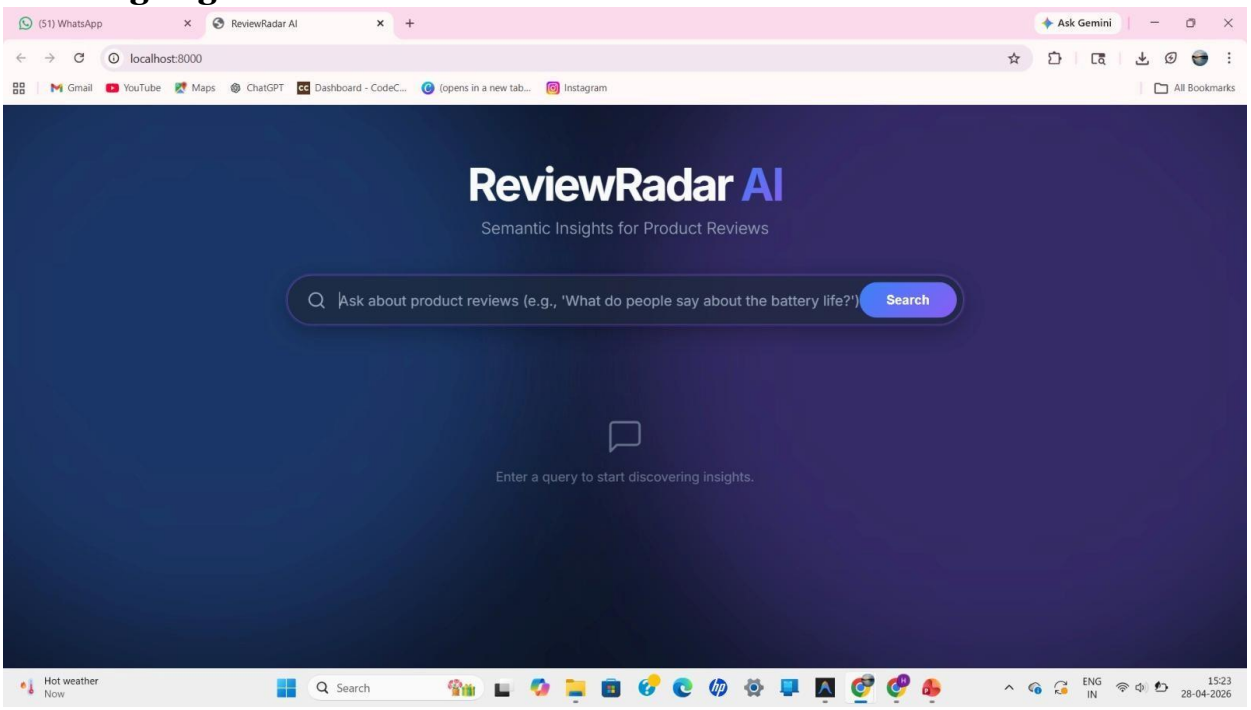

Figure 18: Radar Review AI Landing Page

Input Section with Notes & Controls:

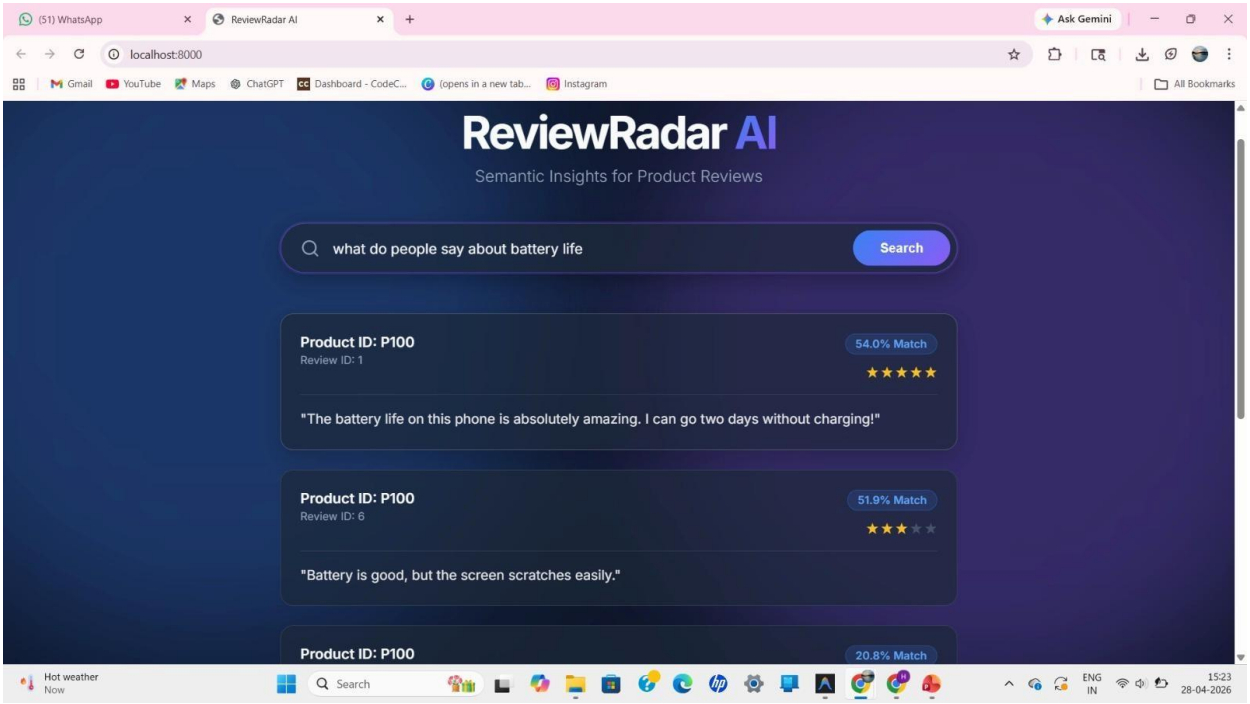

Figure 19: Meeting Notes Input Section **Action**

Items Table:

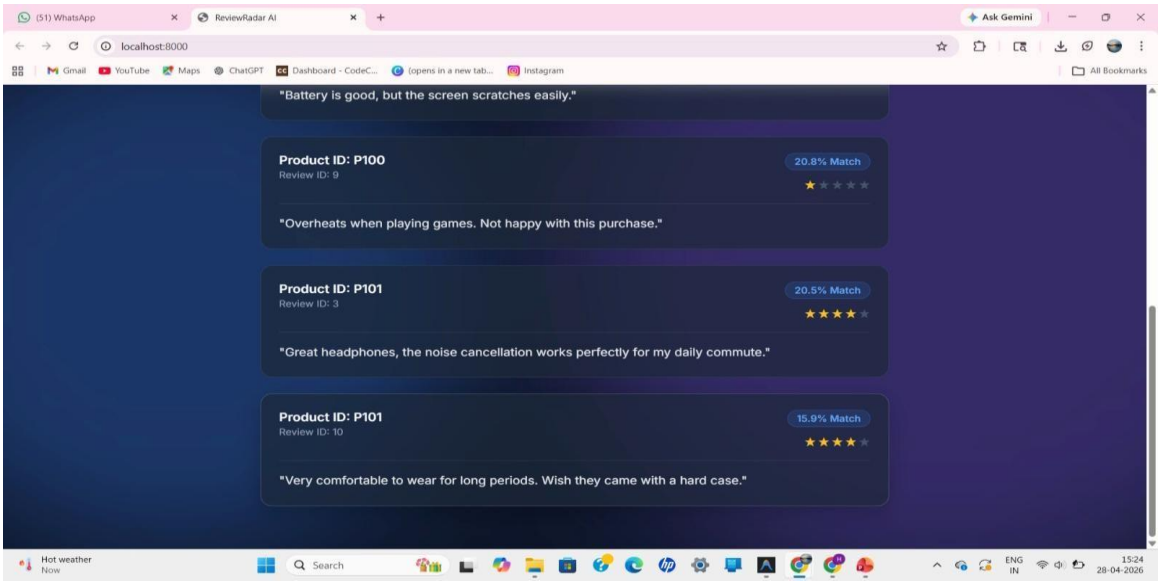

Figure 22: Extracted Action Items with Assignments and Priorities

User Interface & Experience

The system provides a simple and intuitive interface:

- Upload dataset option
- Query input box
- Results display section

The UI ensures ease of use for both technical and non-technical users.

Benchmark Results

- Improved retrieval relevance compared to traditional search
- Reduced manual effort in analysis
- Faster decision-making capability

Overall, the system performs efficiently under real-world conditions.

Advantages & Limitations

Advantages

Semantic Understanding

The system understands the meaning of text rather than relying on exact keywords, resulting in more accurate results.

Time Efficiency

Automates the analysis process, significantly reducing manual effort and time consumption.

Scalability

Designed to handle large volumes of data, making it suitable for enterprise-level applications.

Improved Decision-Making

Provides actionable insights that help businesses make informed decisions.

Flexibility

Supports different types of datasets and can be adapted to various domains.

Limitations

Model Dependency

Performance depends heavily on the quality of embedding models used.

Computational Cost

Embedding generation and similarity search require significant computational resources.

Initial Setup Complexity

System setup and configuration may require technical expertise.

Data Dependency

Accuracy depends on the quality and size of the dataset.

Future Enhancements

Scalability Improvements

- Integration with distributed databases
- Support for large-scale cloud deployment
- Optimization for handling millions of reviews

Feature Expansion

- Add sentiment analysis (positive/negative classification)
- Include keyword extraction and summarization
- Provide visualization dashboards (graphs, charts)

Cloud Integration

- Deploy on AWS, Azure, or Google Cloud
- Enable real-time data streaming
- Add monitoring and logging systems

Mobile Integration

- Develop Android and iOS applications
- Enable real-time notifications for insights
- Provide mobile-friendly interface

Automation

- Automated report generation
- Scheduled data analysis
- Integration with business tools (CRM, ERP systems)

Conclusion

Review Radar AI demonstrates the power of Artificial Intelligence in transforming unstructured textual data into meaningful business insights.

By leveraging Natural Language Processing, semantic search, and vector databases, the system overcomes the limitations of traditional keyword-based approaches. It provides accurate, fast, and scalable solutions for analyzing customer feedback.

The project highlights how AI can enhance business intelligence by reducing manual effort, improving efficiency, and enabling data-driven decision-making. With further enhancements, the system has the potential to become a comprehensive analytics tool for various industries.

Appendix

Configuration Files

- **requirements.txt:** Contains all dependencies required to run the project
- **.env file:** Stores configuration settings and environment variables

API Documentation

Key Endpoints

- **POST /upload_data**

  Upload customer review dataset

- **POST /process_query**

  Accept user query and return results

- **GET /results**

  Retrieve processed outputs

- **DELETE /clear_data**

  Reset stored data

GitHub Repository Link

https://github.com/ritisha29/GEN-AI-II-PROJECT